

Avaliação Comparativa das Ferramentas ETL Apache NiFi e Apache Airflow

Lucas Loscheider Reis Muniz

Orientador: Dr. Evandrino Gomes Barros

Centro Federal de Educação Tecnológica de Minas Gerais – CEFET-MG

Departamento de Computação

Curso de Engenharia de Computação

Abstract

Big Data is an area of Computer Science focused on processing large volumes of data, providing support for more accurate decision-making processes in public and private organizations. In this context, ETL (Extract, Transform, and Load) processes enable the integration of data from multiple sources into a Data Warehouse (DW), a centralized repository for storing historical and updated information for analytical purposes. In this work, the ETL process was structured using an architecture composed of a PostgreSQL data warehouse database used for ingestion and analysis. The overall objective was to comparatively evaluate two free ETL tools—Apache NiFi and Apache Airflow—considering execution efficiency, RAM consumption, CPU usage, and total processing time. The results showed that Apache NiFi was faster in executing flows due to its internal parallelism. However, in terms of system resource consumption, usability, and pipeline maintenance, Apache Airflow performed better. In summary, both tools offer high capacity for implementation and automation of data pipelines, each with specific advantages depending on the type of analytical demand.

Keywords: Big Data. Apache NiFi. Apache Airflow. Scalability.

Resumo

Big Data constitui uma área da Ciência da Computação voltada ao tratamento de grandes volumes de dados, fornecendo subsídios para processos decisórios mais precisos em organizações públicas e privadas. Nesse contexto, processos de ETL (Extract, Transform and Load) viabilizam a integração de dados provenientes de múltiplas fontes em um Data Warehouse (DW), repositório centralizado destinado ao armazenamento de informações históricas e atualizadas para fins analíticos. No presente trabalho, o processo de ETL foi estruturado utilizando uma arquitetura composta por um banco de data warehouse PostgreSQL usado para a ingestão e análise. O objetivo geral consistiu em avaliar comparativamente duas ferramentas gratuitas de ETL — Apache NiFi e Apache Airflow — considerando eficiência de execução, consumo de memória RAM, uso de CPU e tempo total de processamento. Os resultados demonstraram que o Apache NiFi apresentou maior rapidez na execução dos fluxos, em razão de seu paralelismo interno. Entretanto, no que se refere ao consumo de recursos do sistema, usabilidade e manutenção dos *pipelines*, o Apache Airflow apresentou desempenho superior. Em síntese, ambas as ferramentas oferecem elevada capacidade de implementação e automação de *pipelines* de dados, cada uma com vantagens específicas conforme o tipo de demanda analítica.

Palavras-chave: Big Data. Apache NiFi. Apache Airflow. Escalabilidade.

1 Introdução

Com o surgimento da internet, as pessoas passaram a ter acesso a uma enxurrada de informações diariamente, o que contribuiu para um aumento considerável de dados a serem analisados, processados e armazenados.

Dessa forma, surgiu uma nova área dentro da Ciência da Computação, conhecida como Big Data, que se caracteriza como uma abundante quantidade de dados. Devido à sua complexidade, não é possível gerenciar e processar com as ferramentas de processamento tradicionais (OLIVEIRA et al., 2022).

Diante disso, há a necessidade de se realizar o tratamento dessas quantidades volumosas de dados, culminando no surgimento do processo de ETL (*Extract, Transform and Load*). Esse processo realiza a Extração, Transformação e Carregamento de dados de múltiplas fontes (GALDINO, 2016) em um Data Warehouse (DW), que consiste em um repositório centralizado responsável por armazenar dados atuais e históricos de várias fontes, para fins de análise e tomada de decisões. Em seguida, mediante a utilização de um ambiente de visualização, os usuários podem analisá-los e extrair informações úteis para as tomadas de decisões, dentro do mundo corporativo (GALDINO, 2016).

Sendo assim, o objetivo geral deste trabalho foi realizar uma análise comparativa entre o desempenho do Apache NiFi e o Apache Airflow, ambas ferramentas de ETL,

considerando o tempo total dos fluxos de ETL, a porcentagem de uso do processador e a quantidade de memória RAM utilizada durante a execução dos fluxos, bem como uma avaliação de usabilidade das ferramentas.

A realização desse trabalho justifica pela quantidade volumosa de dados e de fontes variadas e que poderão auxiliar as organizações na tomada de decisões. Nesse sentido, o presente trabalho contribui para demonstrar as viabilidades da utilização das ferramentas Apache NiFi e Apache Airflow na realização das etapas de extração, transformação, limpeza e carga desses dados.

O trabalho usa uma base de dados proveniente do Departamento de Informação e Informática do Sistema Único de Saúde (DATASUS)¹, a partir da qual foram construídos fluxos ETL nas duas ferramentas com o propósito de identificar qual delas tem os melhores resultados. O motivo da escolha dessa base de dados se deu em razão de que o DATASUS disponibiliza seus bancos de dados e tabelas para uso livre.

Sendo assim, o presente trabalho foi estruturado em cinco seções, além da introdução e conclusão. Na primeira seção, a de Referencial Teórico, são abordados os conceitos mais importantes para o entendimento da pesquisa realizada como: *Big Data*, Bancos de Dados, PostgreSQL, Apache Airflow e Apache NiFi. Na segunda seção, a de Trabalhos Relacionados, são apresentadas pesquisas na área de ETL que embasaram a elaboração do presente trabalho. Na terceira seção, a de Metodologia, são demonstrados todos os passos necessários para a realização do trabalho. Na quarta seção, a de Resultados, são apresentadas medidas coletadas durante a realização do presente estudo. Na quinta seção, a de Análise, é discutido qual das ferramentas ETL apresentou o melhor desempenho para a realização do processo ETL. Finalmente, na Seção 6, a de Conclusão, são apresentados o resumo do trabalho e os trabalhos futuros.

2 Referencial Teórico

Nesta seção são apresentados os conceitos mais importantes para o entendimento do trabalho.

2.1 Big Data

Big Data é uma nova terminologia usada com o propósito de descrever um enorme volume de dados² desestruturados oriundos das organizações (GOCHI, 2023). Essa abundante quantidade de dados é proveniente de fontes diversas, tais como: planilhas eletrônicas, bancos de dados, registros de venda de produtos, etc. e podem ajudar nas tomadas de decisões assertivas, quando processados adequadamente. Contudo, o grande

¹ Disponível em: <<https://datasus.saude.gov.br/sobre-o-datasus>>. Acesso em: 19 nov. 2025.

² Dados são informações brutas que precisam ser processados para gerar informação.

desafio atual é transformar todos esses dados recebidos em informação, com o propósito de gerar conhecimento e agregar valor às organizações (ERICKSON S.; ROTHBERG, 2014).

2.2 O banco de dados PostgreSQL

Um banco de dados (BD) é um conjunto de dados armazenados de maneira que possam ser manipulados. Sendo assim, um BD é o local onde dados são guardados e organizados para uma utilização posterior (BAZZI, 2013).

Dessa forma, PostgreSQL nada mais é do que mais uma alternativa existente de BD que vem ganhando espaço dentro do mercado em razão de ser de código aberto, possuir alta confiabilidade, melhores recursos de consulta e mais operações previsíveis (CARVALHO, 2017).

Dentre as suas principais funcionalidades, vale destacar: suporte multiplataforma; possibilidade de ser executado em ambiente Windows, Linux e Unix; possuir atomicidade, consistência, isolamento e durabilidade (ACID)³ em todas as suas configurações; Controle de Concorrência Multiversão (MVCC)⁴ (NETO, 2007).

Sendo assim, PostgreSQL é uma alternativa muito eficiente para o armazenamento de quantidades volumosas de dados, sendo utilizado como referência neste trabalho.

2.3 A modelagem multidimensional dos dados

Os bancos de dados relacionais são estruturas que armazenam dados distribuídos em diversas tabelas que se relacionam por meio de campos comuns (SILVA; SARTORI, 2015). Contudo, em se tratando do DW⁵, a modelagem multidimensional é a mais utilizada porque a quantidade de dados armazenada é muito grande e as consultas devem ser mais rápidas e precisas (ANTONIO, 2006). Pensando nisso, por exemplo, a modelagem multidimensional foi utilizada neste trabalho, pois permite consultas analíticas (como, por exemplo, quantidades, somas e contagens) mais rápidas, se comparadas à modelagem não multidimensional.

O modelo multidimensional é composto por uma tabela que possui uma chave composta, conhecida por tabela de fatos e um conjunto de tabelas menores denominadas por tabelas de dimensão com chaves simples (KIMBALL, 1998). Nesse sentido, as dimensões são coleções de atributos descritivos que se correlacionam entre si. Esses atributos são responsáveis por descrever os dados (informações como quem, o quê, quando e onde) e são organizados em dimensões. Já a tabela de fatos sintetiza o relacionamento entre as diversas dimensões

³ São propriedades do banco de dados que garantem que ele permanecerá em um estado válido mesmo após erros inesperados.

⁴ Recurso avançado de banco de dados que cria cópias duplicadas de registros para ler e atualizar com segurança os mesmos dados em paralelo

⁵ Data Warehouse (DW) é um repositório centralizado responsável por armazenar dados atuais e históricos de várias fontes, para fins de análise e tomada de decisões.

(SILVA; SARTORI, 2015), registrando métricas, como, por exemplo, quantidades, somas, médias e desvios padrões.

Portanto, o modelo dimensional é de grande importância para projetos de análise de dados e apoio à decisão, pois permite uma estruturação mais objetiva, consistente e adequada às necessidades analíticas das organizações.

2.4 O Data Warehouse (DW)

O DW consiste em um repositório central de dados que tem a função de agregar informações de um ou mais BDs e mesmo, de outras fontes de informações, para posteriormente realizar o tratamento, formatação e consolidação desses dados (FERREIRA J.; MIRANDA, 2010).

Nesse sentido, o DW é um banco de dados com bastante capacidade de armazenamento e organização que armazena, tanto dados brutos, quanto dados já tratados e consolidados. Dentro do DW ainda há a possibilidade de subdividi-lo em Data Marts (DM), que nada mais são do que pequenas repartições dentro do DW que funcionam como BDs menores e que podem ser utilizados para o armazenamento de informações específicas (GARCIA, 2020).

Já um Data Lake (Lago de dados) é um grande repositório centralizado que armazena grandes volumes de dados brutos, ou seja, dados que ainda não passaram por tratamento. Esse repositório possui alta escalabilidade de armazenamento e é geralmente constituído através da plataforma Hadoop. Contudo, ao contrário do DW que utiliza apenas aplicações pré-definidas, o Data Lake pode ser consultado por aplicações analíticas dinâmicas (GARCIA, 2020).

Conforme o autor, a maior diferença entre o DW e o Data Lake está relacionada com a velocidade de disponibilização dos dados, após eles terem sido gerados nos sistemas que alimentam o lake. Enquanto no DW os dados passam pelo processo ETL, antes do armazenamento, no Data Lake, esse processo de tratamento só ocorre quando os dados são utilizados.

Dentre as suas principais características do DW, vale destacar a centralidade dos dados através da integração de diferentes sistemas como, por exemplo, vendas e finanças em um local único, fornecimento de dados para relatórios e melhora da qualidade dos dados por processos de extração e limpeza (SOUZA, 2021).

A modelagem multidimensional organiza o DW de forma que seja fácil analisar informações sob diferentes perspectivas (tempo, localização, produto, cliente etc.). Nessa abordagem, as tabelas fato armazenam medidas numéricas e eventos que queremos analisar — como vendas, quantidade, valores, duração ou ocorrências. Elas representam o que de fato aconteceu. Já as tabelas dimensão contêm os atributos descritivos que dão contexto às medidas, por exemplo, dados sobre clientes, produtos, datas, regiões ou categorias. Elas representam como ou em que contexto o fato aconteceu. Assim, a combinação entre fato

e dimensões permite criar análises rápidas, comparações, filtros e agregações de forma eficiente, o que é essencial para o *Business Intelligence*⁶.

2.5 O processo de ETL

O processo de ETL é responsável por extrair, transformar e carregar grandes volumes de dados, provenientes de diferentes tipos e formatos. Durante esse processo, os dados são padronizados por meio de algoritmos e podem ter suas relações e correlações identificadas. (GALDINO, 2016).

Na etapa da extração, os dados brutos são obtidos de diversas fontes, como, por exemplo, arquivos CSV⁷, planilhas eletrônicas, outros bancos de dados relacionais e não relacionais. Na fase de transformação, esses dados serão limpos, consolidados e padronizados de modo a garantir a qualidade e confiabilidade dos mesmos. Na carga, esses dados são carregados em um novo sistema de destino, que pode ser um DW (GALDINO, 2016).

Sendo assim, o processo ETL é de grande importância para a realização de tratamento de dados volumosos provenientes de fontes variadas e muito importantes para projetos DW.

2.6 O Apache NiFi

O Apache NiFi é uma ferramenta de código aberto destinada à automação de fluxos de dados entre sistemas de maneira escalável e confiável. Sua abordagem para a condução de processos de ETL fundamenta-se em um modelo visual orientado a componentes, no qual o processamento é configurado por meio da seleção e interconexão de processadores⁸, e não pela escrita direta de código. A partir de sua interface gráfica baseada em navegador, o NiFi permite gerenciar e monitorar os fluxos de dados, possibilitando a extração, transformação e envio de informações entre diferentes fontes e destinos, conforme ilustrado na Figura 1 (CRICKARD, 2020). O fluxo mínimo no Apache NiFi consiste em receber dados de uma fonte *Input Data*, processá-los por meio de componentes configurados *Processor* e encaminhá-los ao destino final de forma automatizada e controlada *Output Data*.

Essa plataforma é altamente tolerante a falhas e projetada para operar em escala, o que a torna adequada para cenários com grande volume de dados. Por meio dos *data flows*⁹, fluxos de dados criados pelos usuários, é possível definir *data flows* complexos que aplicam processamentos avançados, conforme as necessidades do usuário. Graças à sua

⁶ Conjunto de práticas, tecnologias e processos usados para coletar, integrar, analisar e transformar dados brutos em informações úteis para apoiar a tomada de decisão nas organizações

⁷ Arquivo de texto plano que armazena dados tabulares como planilhas. Cada linha representa um registro e cada um dos campos contém valores separados por vírgula.

⁸ Unidades funcionais responsáveis por operações específicas de ingestão, transformação ou roteamento de dados.

⁹ Percurso organizado que os dados seguem desde sua origem até seu destino final, passando por etapas de coleta, transformação e entrega

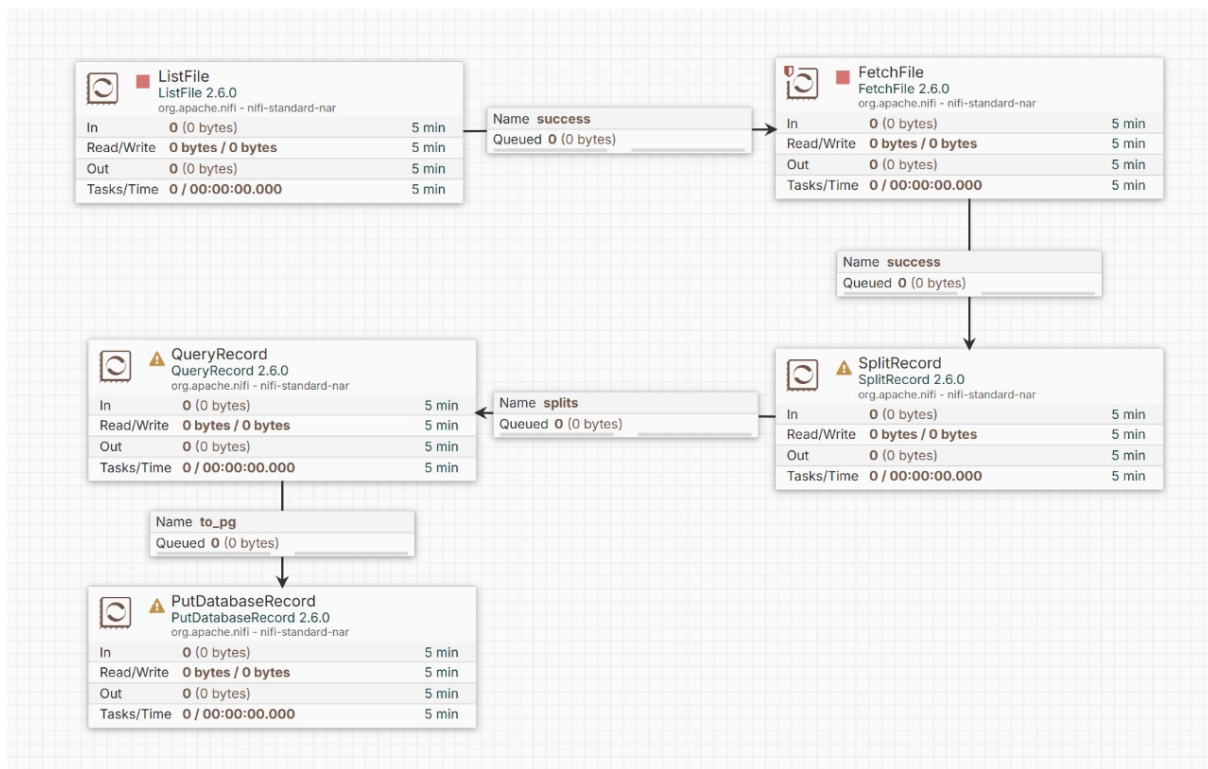


Figura 1 – Fluxo de processamento do grupo sia
Fonte: Elaboração do autor (2025)

flexibilidade, escalabilidade e recursos avançados, o NiFi se destaca como uma solução poderosa para lidar com fluxo de dados em ambientes complexos, sendo muito utilizado em DW (VASCONCELOS, 2024).

No contexto da Engenharia de Dados, o NiFi foi projetado para atuar como uma plataforma de orquestração de fluxos de dados orientada a streaming¹⁰, permitindo que dados sejam coletados, transformados e entregues a diferentes destinos de forma escalável, tolerante a falhas e altamente configurável. Dessa forma, o Apache NiFi é amplamente utilizado como camada de ingestão e integração de dados, sendo especialmente indicado para pipelines que demandam alta taxa de transferência, baixa latência, elasticidade e monitoramento visual contínuo dos fluxos de dados(CRICKARD, 2020).

2.7 Apache Airflow

O Apache Airflow é considerado um orquestrador de dados¹¹, cuja estrutura permite criar fluxos de dados, orientados em lote. Seu principal recurso é uma plataforma capaz de construir *pipelines* ou *data flows* de dados (GUARDIANI et al., 2022).

Ainda, segundo o autor, o Apache Airflow consegue gerenciar fluxos onde a saída de um DG alimenta a entrada de outro DAG, estabelecendo uma sequência de processamento.

¹⁰ Processamento ou transmissão de forma contínua, à medida que esses dados são gerados, sem esperar o conjunto completo de dados

¹¹ Ferramenta que automatiza ou gerencia fluxos de trabalho, também conhecidos como *pipelines*.

Esse recurso é conhecido como gerenciamento de dependências cruzadas entre DAGs (*cross-DAG dependencies*).

Seu funcionamento ocorre por meio da criação de um *Directed Acyclic Graph (DAG)*¹² que gera uma coleção de tarefas organizadas, executadas em uma ordem determinada. A Figura 4 ilustra um exemplo de um DAG no Apache Airflow.

Todos os fluxos são definidos em linguagem Python, sendo flexíveis e acessíveis para desenvolvimento. Além disso, há uma interface gráfica acessível via navegador que permite visualizar e gerenciar DAGs, facilitando o monitoramento e a resolução de problemas (FRANZONI, 2020), como pode ser visto na Figura 2.

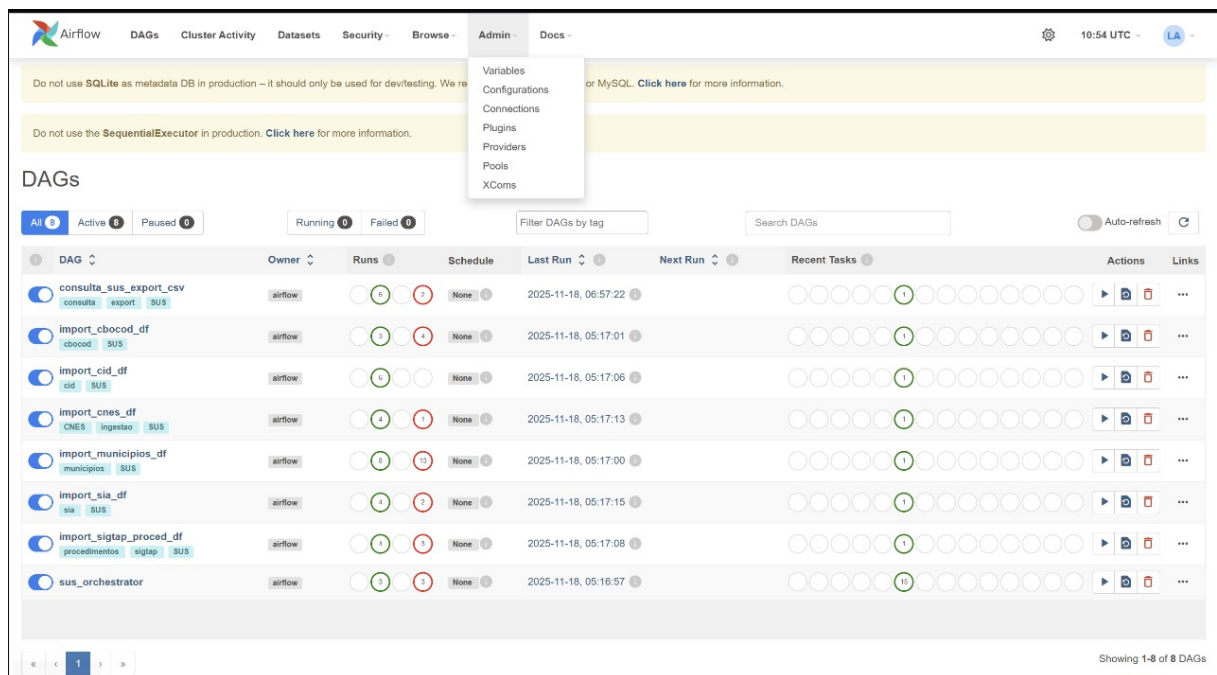


Figura 2 – Tela do Apache Airflow
Fonte: Elaboração do autor (2025)

O Apache Airflow apresenta os seguintes componentes principais: *scheduler*, *workers* e *webserver*. O *scheduler* tem a função de definir a agenda e o intervalo entre as execuções do *pipeline*; os *workers* são os responsáveis pela execução do código Python para a realização das tarefas; e o *webserver* é a interface da aplicação que permite o monitoramento e a auditoria da execução dos fluxos (GUARDIANI et al., 2022). Dessa forma, o Apache Airflow é uma plataforma robusta, confiável e escalável para trabalhar com a transferência e o monitoramento de dados enviados em lote e periodicamente.

No contexto da Engenharia de Dados, o Airflow atua como uma ferramenta de automação e coordenação de processos ETL, especialmente em cenários batch¹³ e periódicos. Sendo assim, o Airflow não é projetado para ingestão de dados em tempo real, mas sim

¹² DAG - Grafo Acíclico Dirigido, um tipo de gráfico usado em Computação e Matemática.

¹³ situações em que os dados são processados em blocos, também conhecidos como lotes em momentos específicos, e não de forma contínua

para coordenação de tarefas computacionais, como execuções de *scripts*, cargas em bancos de dados, chamadas a APIs, treinamentos de modelos e integrações com serviços externos. Assim, o Apache Airflow é amplamente adotado como uma camada de orquestração e automação de *pipelines* de dados, sendo mais indicado para cenários em que a previsibilidade, reprodutibilidade e governança dos fluxos ETL são mais relevantes do que a taxa de ingestão ou o processamento contínuo de dados (FRANZONI, 2020).

3 Trabalhos Relacionados

Nesta seção são apresentados alguns trabalhos na área de Big Data que apresentam fluxos ETL para armazenamento de dados em um DW e que motivaram a realização do presente trabalho.

Contudo, a escassez de pesquisas que focavam na comparação das mesmas ferramentas utilizadas neste trabalho se deu em razão das diferenças conceituais, arquiteturais e funcionais entre as duas ferramentas, projetadas para atender a propósitos distintos dentro da Engenharia de Dados.

Dessa forma, as pesquisas que mostram a comparação direta entre as duas ferramentas ETL apresentadas neste trabalho, foram muito incipientes devido à dificuldade metodológica de estabelecer métricas de comparação equivalentes entre ferramentas que operam sob modelos de execução distintos. Métricas como tempo de execução, uso de CPU, consumo de memória e taxa de transferência possuem interpretações diferentes em arquiteturas orientadas a fluxo contínuo e em sistemas baseados em execução batch, o que exige cenários experimentais cuidadosamente controlados para garantir comparabilidade e reprodutibilidade.

No entanto, foram encontrados na literatura, trabalhos que apresentam o comparativo de desempenho de ETLs em relação à utilização de outras ferramentas *Big Data* e serão apresentados, a seguir:

No âmbito dos trabalhos que focam na utilização de ferramentas para a realização do processo ETL, é possível citar o trabalho de Silva (2024), que criou um DW para dados de atendimentos ambulatoriais do SUS, utilizando o motor de processamento distribuído do Apache Spark¹⁴. Todos os dados passaram por um processo de ETL e foram armazenados em um modelo dimensional, contendo uma tabela de fatos e 12 tabelas de dimensão. Dessa forma, foi possível realizar consultas analíticas de alta complexidade, retornando os resultados em tempo hábil.

Além disso, no trabalho de Souza (2020), criou-se um repositório unificado de dados de saúde pública na forma de um DW com carga de dados por ETL. O esquema de dados multidimensionais integra dados de três fontes na Secretaria Municipal de São Paulo:

¹⁴ Framework de processamento distribuído que permite analisar grandes volumes de dados com alta velocidade

SIM (Sistema de Informações sobre Mortalidade); SINASC (Sistema de Informações de Nascidos Vivos) e SIHSUS (Sistema de Informações Hospitalares do SUS). O motor de processamento utilizado foi o Apache Airflow, combinado com o BD PostgreSQL. O resultado evidenciou uma redução do tempo de processamento e do consumo de memória RAM ao processar uma fonte massiva contendo 45.592.442 milhões de registros.

Também é importante citar o trabalho de PAGOTTO D. P.; MARQUES (2024) sobre a construção de um *data lake* com dados do setor de saúde para armazenamento, sistematização e disponibilização destes dados sobre a saúde brasileira. Através das ferramentas Apache Airflow e Dremio¹⁵, as bases de dados do DATASUS passam por um processo ETL com o propósito de realizar a extração, carregamento e tratamento dessas bases para o *data lake*. Os resultados evidenciaram a capacidade da plataforma armazenar grandes volumes de dados, bem como propiciar uma navegação intuitiva, facilitando a compreensão e o manuseio dos dados por analistas em saúde.

Outro trabalho que merece ser apresentado é o de (JESUS; CAMARGOS; SOUZA, 2023) O qual propõe uma análise comparativa das ferramentas ETL: Apache Airflow e Apache NiFi, mediante a utilização da plataforma BigQuery da Google, que opera em um ambiente de computação em nuvem, onde os dados provenientes de arquivos CSV, disponíveis na internet foram armazenadas para compor a carga dos processos ETL. O objetivo dessa comparação foi definir o comportamento dessas duas ferramentas na extração de diversas fontes de dados, capacidade de exibir pré-visualização dos dados, tempo gasto para a realização dos fluxos ETL, tempo de consumo de CPU e acesso à memória RAM. Os resultados evidenciaram que o Apache Nifi apresentou resultados mais satisfatórios em quesitos como: obtenção de dados de fontes distintas e pré-visualização dos dados. Contudo, no monitoramento do consumo de CPU e no acesso à memória RAM, o NiFi consome mais recursos do sistema, sendo que, seu consumo de CPU foi 9 por cento a mais do que o Airflow. No consumo de memória RAM, o NiFi chegou a consumir 10Gb de memória processando 1,5Gb de dados. Já o Airflow, não apresentou diferença significativa no consumo de memória ficando em torno de 2 Gb. Em se tratando do tempo gasto para o processamento dos fluxos ETL, o NiFi despontou como mais veloz, gastando em média, 17,1 por cento a menos de tempo para processar 1,5Gb de dados em comparação ao Airflow. Diante disso, o estudo concluiu que o Airflow é uma ferramenta para a automação de tarefas, sendo mais simples e rápida em alguns quesitos. Contudo, o NiFi oferece mais opções nativas para tratamento dos dados, deixando sua execução mais pesada e consumindo mais recursos do sistema.

Por fim, também é importante citar o trabalho de TOLEDO D. F.; CORREIA (2025) sobre Descoberta de Conhecimento em Bases de Dados (*Knowledge Discovery in Databases - KDD*) que tem o intuito de auxiliar os seres humanos na tarefa de analisar esses grandes

¹⁵ Plataforma de virtualização e aceleração de dados que permite consultas analíticas unificadas diretamente sobre diferentes fontes, sem necessidade de mover ou replicar dados. Disponível em: <<https://www.dremio.com>>. Acesso em: 2 dez. 2025.

conjuntos de dados. Sendo assim, o objetivo da pesquisa foi demonstrar as principais características de três ferramentas ETL, destacando suas semelhanças e diferenças e criando uma avaliação para auxiliar futuros trabalhos na área. Dentre as ferramentas ETL analisadas por este estudo, vale destacar: Apache NiFi, Pentaho Data Integration e Power Query. Os resultados evidenciaram que cada uma das ferramentas se adequa a um determinado contexto diferente. Em se tratando de fluxos de dados complexos, por exemplo, o Apache NiFi se destaca por sua escalabilidade e gratuidade de uso. O *Pentaho Data Integration*¹⁶, por sua vez, possui uma interface amigável e recursos abrangentes de transformação de dados. Enquanto, o *Power Query*¹⁷ é adequado para aplicações que já utilizam outros produtos da Microsoft e desejam uma solução integrada para análise de dados e integração de dados.

Contudo, esses trabalhos se diferem do presente em razão de que não mostram o comparativo de desempenho do Apache Airflow com o Apache NiFi em relação ao tempo gasto para a execução dos fluxos ETL e consumo de recursos do sistema como: processamento e memória RAM.

Sendo assim, o presente trabalho contribuirá para identificar quais os principais recursos de uma máquina física são consumidos durante a execução dessas ferramentas, contribuindo para que, especialistas da área, definam quais desses recursos são os mais adequados às suas necessidades e recursos disponíveis.

4 Metodologia

Nesta seção, são apresentados todos os passos seguidos para a realização dos testes nas duas ferramentas ETL analisadas no presente trabalho. A metodologia possui três etapas que são: Obtenção das bases de dados; Construção dos fluxos e Execução dos fluxos, conforme pode ser visto na Figura 3 a seguir.

¹⁶ Disponível em: <<https://pentaho.com/products/pentaho-data-integration/>>. Acesso em: 2 dez. 2025.

¹⁷ Disponível em: <<https://learn.microsoft.com/pt-br/power-query/power-query-what-is-power-query>>. Acesso em: 2 dez. 2025.

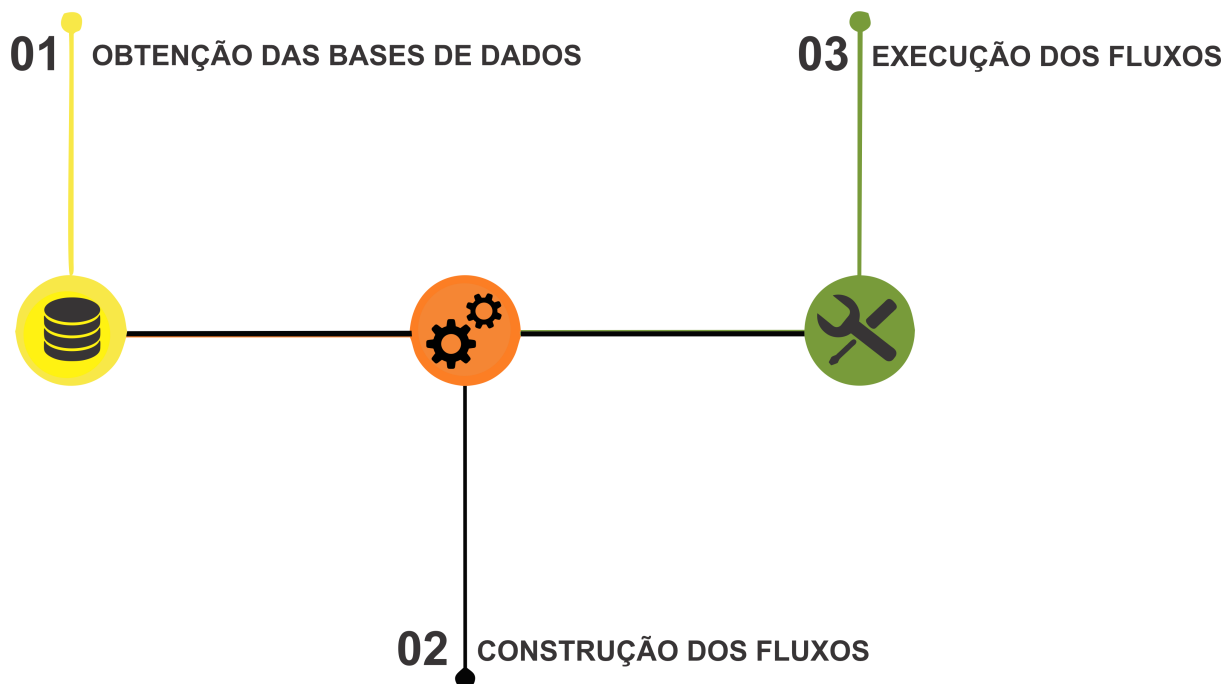


Figura 3 – Ordem da metodologia seguida
Fonte: Elaboração do autor (2025)

No presente trabalho, o fluxo de processamento implementado no Apache Airflow foi concebido de forma estritamente sequencial, respeitando a ordem lógica das etapas de ingestão e transformação dos dados. Essa decisão metodológica foi adotada visando assegurar previsibilidade, reprodutibilidade e controle experimental, aspectos fundamentais em estudos comparativos de desempenho no contexto da Engenharia de Dados. Embora o Apache Airflow ofereça suporte à execução paralela de tarefas independentes, o paralelismo no Airflow ocorre no nível de agendamento de tarefas e não no processamento interno dos dados, diferindo conceitualmente do paralelismo intrínseco presente em ferramentas orientadas a fluxo contínuo, como o Apache NiFi.

4.1 Obtenção das bases de dados

No presente trabalho foi utilizado um banco de dados PostgreSQL que recebeu o nome de SIA e constituiu-se no banco de importação dos dados provenientes do DATASUS e IBGE.

As bases de dados fontes utilizadas no presente trabalho são provenientes do Sistema de Informações Ambulatoriais (SIA) que está disponível para download na página do DATASUS¹⁸. Já as informações sobre municípios brasileiros, foram obtidas no site do Instituto Brasileiro de Geografia e Estatística (IBGE)¹⁹. Estes dados estão disponíveis em tabelas que podem ser baixadas pela internet.

¹⁸ Disponível em: <<https://datasus.saude.gov.br/transferencia-de-arquivos>>. Acesso em: 19 nov. 2025.

¹⁹ Disponível em: <<https://servicodados.ibge.gov.br/api/v1/localidades/>>. Acesso em: 19 nov. 2025.

No banco de dados SIA, foram estruturadas 13 tabelas, sendo uma, a tabela de fatos e 12 tabelas de dimensão conforme pode ser visto na Tabela 1.

Tabela	Descrição	Fato/Dim.	Atributos	Registros	Tamanho
sia_df	Sist. Inf. Ambulato- riais	Fatos	60	42.426.633	9.68 GB
municípios_df	Municípios e Estados	Dimensão	9	853	74.4 KB
cnes_df	Cad. Estab. Saúde	Dimensão	55	3.145.949	992 MB
sigtap_proced_df	Procedimentos	Dimensão	4	3631	246 KB
cid_df	CID	Dimensão	9	12.447	1.31 MB
ano_mes_df	Tabela de datas	Dimensão	5	12	2.66 KB
cbocod_df	Cód. Bras. Ocupações	Dimensão	2	2812	63 KB
tpups_df	Estab. de saúde	Dimensão	2	42	7.55 KB
catend_df	Caráter Atendimento	Dimensão	2	18	7.32 KB
docorig_df	Produção ambulatorial	Dimensão	2	6	5.58 KB
sexo_df	Sexo dos pacientes	Dimensão	2	3	2.64 KB
raca_cor_df	Raça e cor do paciente	Dimensão	2	14	6.65 KB
motsai_df	Motivos de saída	Dimensão	2	22	7.5 KB

Tabela 1 – Tabela multidimensional
Fonte: Elaboração do autor (2025)

Desta base de dados foram importadas as seguintes tabelas: **sia_df**²⁰, **cnes_df**²¹, **sigtap_proced_df**²², **cbocod_df**²³, **cid_df**²⁴.

As tabelas de dimensão: **tpups_df**, **catend_df**, **docorig_df**, **motsai_df**, **ano_mes_df**, **sexo_df** e **raca_cor_df**²⁵, foram obtidas na mesma fonte e construídas com base em

²⁰ Disponível em: <ftp://ftp.datasus.gov.br/dissemin/publicos/SIASUS/200801_/Dados/PAMG2401a.dbc>. Acesso em: 19 nov. 2025.

²¹ Disponível em: <ftp://ftp.datasus.gov.br/dissemin/publicos/CNES/200508_/Dados/ST/STMG2401.dbc>. Acesso em: 19 nov. 2025.

²² Disponível em: <http://sigtap.datasus.gov.br/tabela-unificada/app/sec/inicio.jsp>. Acesso em: 19 nov. 2025.

²³ Disponível em: <ftp://ftp.datasus.gov.br/dissemin/publicos/SIASUS/200801_/Auxiliar/TAB_SIA.zip>. Acesso em: 19 nov. 2025.

²⁴ Disponível em: <http://www2.datasus.gov.br/cid10/V2008/descrcnv.htm>. Acesso em: 19 nov. 2025.

²⁵ Disponível em: <ftp://ftp.datasus.gov.br/dissemin/publicos/SIASUS>. Acesso em: 19 nov. 2025.

arquivos auxiliares de tabulação disponibilizados pelo DATASUS. Essas tabelas não apresentavam variabilidade temporal nem dependiam de processos de atualização contínua, configurando-se como dimensões categóricas simples. Considerando que a criação de fluxos no Apache NiFi ou de DAGs específicas no Apache Airflow para esse tipo de dado implicaria em maior sobrecarga de implementação e manutenção, adotou-se a estratégia de construir manualmente tais tabelas no banco de dados. Essa decisão permitiu reduzir o custo operacional do fluxo de dados sem afetar a completude do modelo analítico. Dessa forma, todas essas tabelas criadas manualmente, fizeram parte do fluxo ETL com o propósito de contribuir com as medições de tempo e desempenho das ferramentas analisadas.

A importação dos dados para o BD SIA, se deu mediante a realização de um processo de conversão de formato dessas tabelas, pois todos os dados estavam criptografados no formato DBC²⁶. Com a utilização do aplicativo *Tabwin*,²⁷ foi possível converter as tabelas que estavam armazenadas em um diretório do projeto para o formato DBF²⁸. Posteriormente, estes dados foram convertidos em formato CSV e apresentaram alguns erros, como, por exemplo, caracteres especiais e variáveis com tipos diferentes do tipo definido para determinada coluna da tabela. Após realizar a correção desses dados, eles foram rearmazenados em novos arquivos CSV e foram importados normalmente para o BD SIA.

4.2 Construção dos fluxos

A importação dos dados para o BD SIA ocorreu mediante os dados que estavam contidos nos novos arquivos CSV rearmazenados no diretório do projeto. O BD SIA concentra o modelo analítico e é o ambiente no qual foi executada a consulta SQL a qual buscou verificar, qual é a quantidade de procedimentos e valores aprovados pelo SUS, durante os meses de janeiro, fevereiro, março e abril de 2024, realizados em unidades básicas em pacientes negros e do sexo masculino, agrupados por Região do País, mês e subcategoria do CID, conforme apresentada a seguir:

```
SELECT
    municipios_df.regiao_sigla,
    ano_mes_df.ano,
    cid_df.co_categ,
    cid_df.no_categ,
    SUM(sia_df.pa_qtdapr) AS procedimentos,
    SUM(sia_df.pa_valapr) AS valor
FROM sia_df
```

²⁶ Formato de arquivo de dados compactado usado pelo DATASUS

²⁷ Disponível em: <<ftp://ftp.datasus.gov.br/dissemin/publicos/SIASUS>>. Acesso em: 19 nov. 2025.

²⁸ Formato de arquivo de banco de dados padrão, conhecido como dBase, que organiza dados em registros (linhas) e campos (colunas)

```

JOIN municipios_df
    ON sia_df.pa_ufmun = municipios_df.pa_ufmun
JOIN ano_mes_df
    ON sia_df.pa_cmp = ano_mes_df.pa_cmp
JOIN cnes_df
    ON sia_df.pa_coduni = cnes_df.pa_coduni
JOIN sigtap_proced_df
    ON sia_df.pa_proc_id = sigtap_proced_df.pa_proc_id
    AND sia_df.pa_cmp = sigtap_proced_df.dt_competencia
JOIN cid_df
    ON sia_df.pa_cidpri = cid_df.co_cid
JOIN sexo_df
    ON sia_df.pa_sexo = sexo_df.pa_sexo
JOIN raca_cor_df
    ON sia_df.pa_racacor = raca_cor_df.pa_racacor
WHERE cnes_df.tp_unidade_desc ILIKE '%UNIDADE BASICA%'
    AND sigtap_proced_df.no_procedimento ILIKE '%PSICOTERAPIA%'
    AND ano_mes_df.ano IN ('2018', '2019', '2020')
    AND sexo_df.sexo_desc = 'Masculino'
    AND raca_cor_df.racacor_desc = 'PRETA'
GROUP BY
    municipios_df.regiao_sigla,
    ano_mes_df.ano,
    cid_df.co_categ,
    cid_df.no_categ
ORDER BY
    municipios_df.regiao_sigla ASC,
    ano_mes_df.ano ASC,
    SUM(sia_df.pa_valapr) DESC;

```

A escolha do PostgreSQL ocorreu por ser um banco de dados de código aberto, de fácil utilização e por utilizar comandos SQL; além disso, é robusto no armazenamento de quantidades volumosas de dados (SOUZA; AMARAL; LIZARDO, 2011).

Foi necessária a instalação e configuração do WSL Ubuntu versão 24.04 no sistema operacional Windows 11, a fim de emular um ambiente Linux. A escolha desse sistema operacional ocorreu em razão de que, o Apache NiFi apresenta um melhor desempenho e estabilidade no Linux, do que no sistema operacional Windows. Já, em se tratando do Apache Airflow, não há nenhum suporte dessa ferramenta para o Windows.

De acordo com Yang e Vetter (2021) Optou-se pela utilização do Ubuntu como sistema operacional para a execução do Apache NiFi, uma vez que esse ambiente oferece maior

estabilidade, melhor desempenho para aplicações baseadas em Java e compatibilidade nativa com os serviços e utilitários do sistema operacional Linux. Tais características garantem uma operação mais eficiente e confiável dos fluxos de ingestão de dados quando comparada à execução em sistemas Windows.

No caso do Apache Airflow, a adoção do Ubuntu também se mostrou mais adequada, pois esse sistema operacional apresenta maior compatibilidade com o ecossistema Python e com os serviços Unix utilizados pelo orquestrador, além de oferecer melhor desempenho, estabilidade e facilidade de gerenciamento dos processos em segundo plano. Dessa forma, sua execução em ambiente Linux tende a ser mais eficiente e confiável do que em sistema Windows. Logo em seguida, foi necessário instalar e configurar as duas ferramentas: Apache Airflow e Apache NiFi, visando realizar os testes dos processos ETL. A escolha dessas ferramentas de código aberto se deu em razão de que estão entre as mais utilizadas no mundo corporativo para aplicações de Big Data (REIS; NAKAOKA; NETO, 2023).

No sistema operacional Ubuntu, instalou-se o Python, versão 3.12, utilizado pelo Airflow e também um arquivo JDBC (Java Database Connectivity), que é uma API do Java e permite que aplicações escritas em Java, no caso o NiFi, se conectem ao banco PostgreSQL.

O processo de ETL foi estruturado de modo a permitir a importação e o processamento integral do conjunto de dados provenientes dos novos arquivos CSV previamente reorganizados. Após a carga, o banco de dados SIA serviu como base para a execução da consulta SQL utilizada. Essa mesma base foi empregada nos experimentos de desempenho, nos quais foram mensurados o percentual de uso do processador, a quantidade de memória RAM consumida e o tempo total necessário para a conclusão dos fluxos de ETL.

4.2.1 Construção do Fluxo de Dados no Airflow

O Apache Airflow executou as etapas de transformação por meio de *scripts* em Python e organizou o *pipeline* em Grafos Acíclicos Dirigidos (DAGs). Em cada *pipeline*, as tarefas foram definidas como operações que acionam *scripts* Python, responsáveis pela execução de cada etapa.

No presente trabalho, os DAGs utilizados foram armazenados nos seus respectivos arquivos import do projeto e fazem referência às tabelas: `sia_df`, `cnes_df`, `sigtap_proced_df`, `cbocod_df`, `cid_df` e `municipios_df`. Todas essas tabelas foram criadas dentro do BD SIA. Em seguida, os DAGs são executados, realizando o “IMPORT” de cada uma das 6 tabelas. Depois, um DAG chamado `consulta` realiza a consulta SQL apresentada anteriormente. A Figura 4 ilustra como o DAG da tabela dimensão `cnes_df` foi criada em linguagem Python.


```

from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime
from scripts.ingest_csv import load_csv_to_postgres

# =====
# Caminhos e parâmetros específicos da DAG CNES
# =====
CSV_PATH = "/mnt/c/Users/lucas/documents/TCC/tcc3/tabelas_padronizadas/cnes/CNES_padronizado.csv"
TABLE_NAME = "cnes_df"

# =====
# Definição da DAG
# =====
with DAG(
    dag_id="import_cnes_df",
    description="Importa o arquivo CNES.csv para a tabela cnes_df no PostgreSQL",
    start_date=datetime(2025, 10, 10),
    schedule_interval=None, # execução manual
    catchup=False,
    tags=["SUS", "CNES", "ingestao"],
) as dag:

    ingest_cnes = PythonOperator(
        task_id="ingest_cnes",
        python_callable=load_csv_to_postgres,
        op_kwargs={
            "csv_path": CSV_PATH,
            "table_name": TABLE_NAME
        },
    )

    ingest_cnes

```

Figura 4 – Código python do DAG referente à tabela de dimensão cnes_df
 Fonte: Elaboração do autor (2025)

Foram configurados parâmetros essenciais, como o identificador (*dag_id*), a descrição do fluxo, a data de início e o intervalo de execução, que determinam como o Airflow irá agendar e gerenciar o processo. Em seguida, foram definidas as tarefas que compõem o fluxo, cada uma representada por um operador que encapsula uma ação específica. No exemplo da Figura 4, utiliza-se um *PythonOperator*, responsável por executar uma função que realiza a ingestão de um arquivo CSV no BD SIA, recebendo para isso um nome (*task_id*), que é a função a ser chamada e seus argumentos.

O fluxo de ETL todo é criado mediante um arquivo chamado de *orchestrator.py*, o qual reúne todos os *dags* para formar um fluxo único no painel do Apache Airflow, conforme pode ser visto na Figura 5.

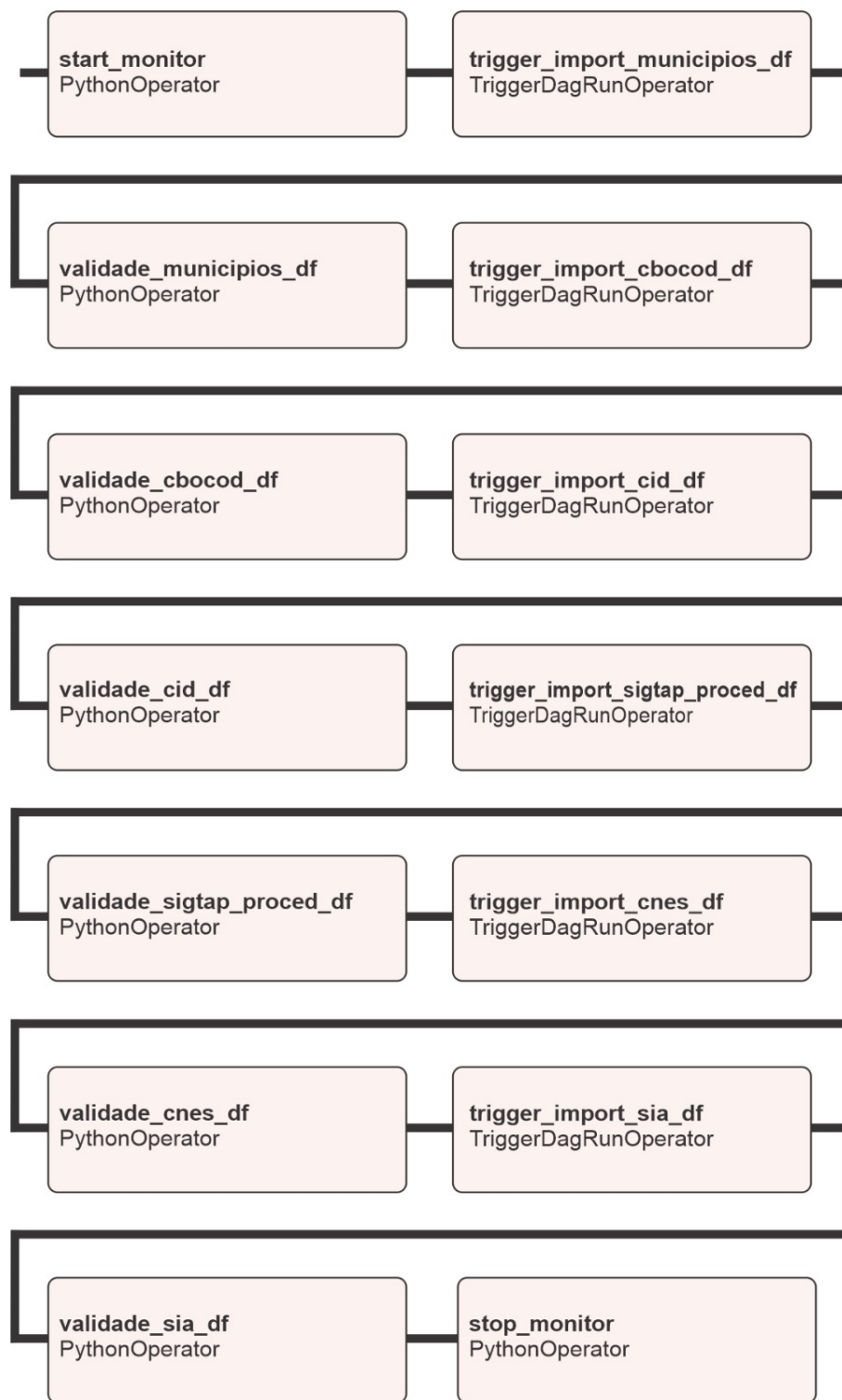


Figura 5 – Fluxo de ETL do Apache Airflow

Fonte: Elaboração do autor (2025)

4.2.2 Construção do Fluxo de Dados no NiFi

No Apache NiFi, o fluxo ETL é criado mediante seis grupos de processadores. Para cada um desses grupos de processadores, apresentados na Figura 6, há um fluxo de processamento próprio.

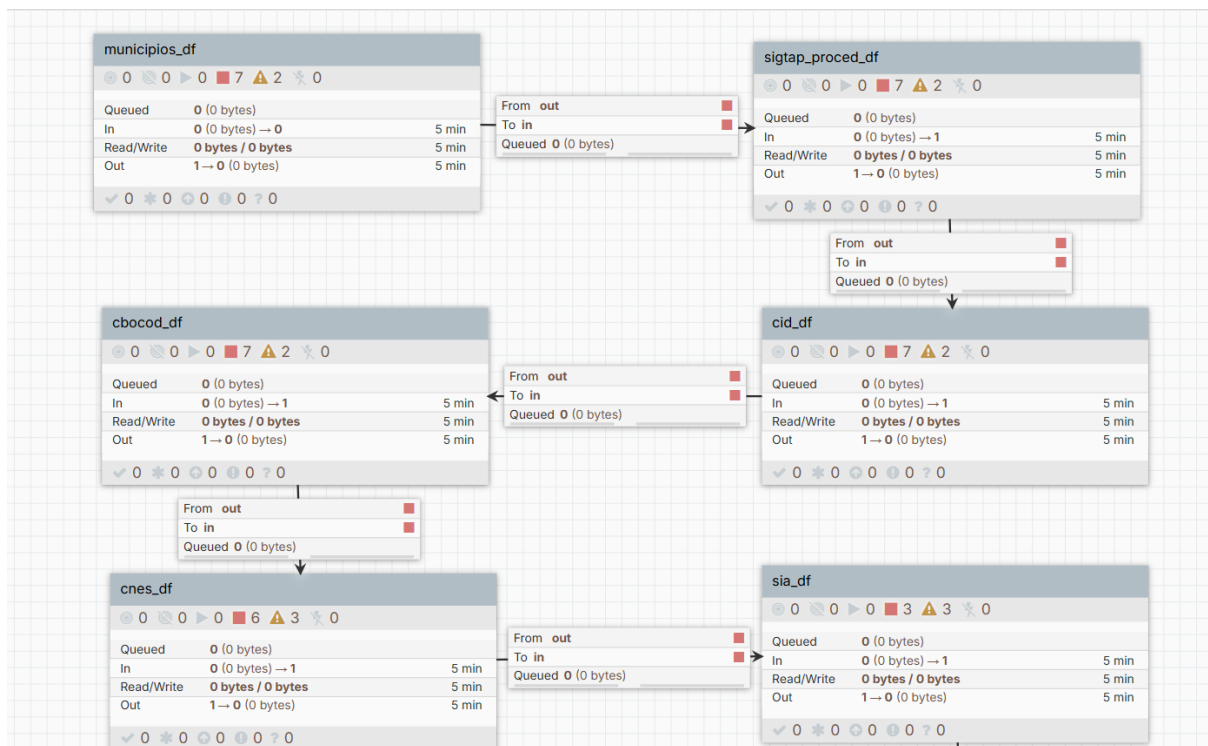


Figura 6 – Grupo de processadores NiFi

Fonte: Elaboração do autor (2025)

Os seis grupos de processadores utilizados no fluxo ETL do NiFi são:

- Grupo 1 – `municipios_df`;
- Grupo 2 – `sigtap_proced_df`;
- Grupo 3 – `cid_df`;
- Grupo 4 – `cbocod_df`;
- Grupo 5 – `cnes_df`;
- Grupo 6 – `sia_df`.

A Figura 1 ilustra claramente a estrutura do sexto grupo de processadores, denominado, `sia_df`. Esse grupo é composto por 5 processadores, sendo o primeiro, chamado de *ListFile*, o qual é responsável por verificar se o diretório, onde o arquivo *municipios.csv* existe no diretório informado ao processador. O segundo processador, chamado de *FetchFile*, é responsável por obter os dados dentro do arquivo *municipios.csv*. O terceiro processador, chamado de *SplitRecord*, é responsável por dividir os dados do arquivo .csv em arquivos menores, para não ocorrer falta de memória no próximo processador, ao recebê-los. Contudo, os outros grupos de processadores não possuem o *Splitrecord* em razão de que os dados das tabelas são pequenos, descartando essa necessidade. O quarto processador, chamado de *QueryRecord*, é responsável por verificar se a quantidade de colunas, o tipo

das variáveis de cada coluna e seus nomes coincidem com os mesmos dados na tabela, correspondente ao BD SIA. O quinto e último processador, de nome *PutDatabaseRecord* é responsável por inserir os dados do arquivo na tabela correspondente no BD SIA.

Terminado este fluxo, o próximo grupo de processadores é executado, sendo esse grupo e os posteriores compostos por uma estrutura de processadores, idênticas à estrutura descrita do grupo de processadores municípios, que é o Grupo 1.

Em relação aos testes de fluxos de ETL, os quais foram desde a obtenção das fontes dos dados do SUS até a carga no BD SIA, foram executados dez vezes em cada uma das duas ferramentas ETL analisadas.

4.3 Execução dos fluxos

É importante dizer que o monitoramento do processador e da memória RAM foi realizado por meio de um *script* desenvolvido em Python (chamado `monitor_python`), com amostragens registradas em intervalos de 5 segundos, utilizando a biblioteca *psutil*, a qual permite acessar métricas de desempenho do sistema operacional (AGUIAR C. G. B.; FARIA JUNIOR, 2024). A Figura 7 ilustra todo o processo, de modo geral e resumido, dos fluxos NiFi e Airflow medidos.



Figura 7 – Fluxo de processamento geral

Fonte: Elaboração do autor (2025)

Primeiramente, o *script* Python `monitor_python` é executado para medir a quantidade de memória RAM e de todo o processamento utilizados durante todo o fluxo do NiFi e do Airflow. Então, o fluxo, seja do Airflow ou do NiFi é executado até o fim e, por último, o *script* Python `monitor_python` é finalizado, armazenando os resultados do desempenho em um arquivo `.csv`.

Foram realizadas 10 medições, sendo que, para o Apache Airflow, houve 1214 repetições de testes com intervalo de 5 segundos cada e a média do tempo de execução total das 10 medições de todo o fluxo foi de 01:41:14 (horas, minutos e segundos, respectivamente). Já para o Apache NiFi, também foram realizadas a mesma quantidade de medições com 468 repetições de testes em intervalos de 5 segundos. A média de tempo de execução total foi de 00:39:28 (horas, minutos e segundos).

No presente estudo, as medições de desempenho referentes ao uso de CPU e memória foram realizadas de forma global no nível do sistema operacional, considerando o consumo total de recursos do ambiente de execução, e não apenas os processos diretamente associados às ferramentas Apache NiFi e Apache Airflow. Essa decisão metodológica foi adotada visando capturar de maneira mais representativa o impacto real de cada ferramenta sobre o sistema computacional na totalidade, refletindo um cenário mais próximo de ambientes produtivos.

5 Resultados e discussão

As duas ferramentas foram analisadas no presente estudo em relação ao tempo gasto para a execução dos fluxos ETL e consumo de recursos do sistema como: processamento e memória RAM. Dessa forma, o comparativo de desempenho das duas ferramentas, pode ser visto na Tabela 2, que também demonstra as médias de CPU e RAM para cada uma das 10 execuções. É perceptível que o NiFi consome mais memória RAM, entre 2.0 a 2.7 GB. Já o Apache Airflow se manteve estável durante a execução da *pipeline*, com consumo, em torno de 2.0 a 2.6 GB.

Tabela 2 – Comparação consolidada das execuções e consumo de recursos das ferramentas ETL

Métrica	Média Airflow	Média NiFi	1 ^a	2 ^a	3 ^a	4 ^a	5 ^a	6 ^a	7 ^a	8 ^a	9 ^a	10 ^a
Tempo total ETL	01:41:14	00:39:00	–	–	–	–	–	–	–	–	–	–
Uso de RAM (GB) – Airflow	2.0–2.6	–	2.0	2.2	2.1	2.4	2.2	2.6	2.5	2.2	2.3	2.4
Uso de CPU (%) – Airflow	3.2–3.7	–	3.5	3.7	3.4	3.2	3.6	3.5	3.2	3.3	3.6	3.5
Uso de RAM (GB) – NiFi	–	2.0–2.7	2.0	2.7	2.3	2.5	2.4	2.6	2.1	2.3	2.2	2.5
Uso de CPU (%) – NiFi	–	4–5	4.0	4.5	4.3	4.7	4.3	4.8	5.0	4.7	4.3	4.4

Fonte: Elaboração do autor (2025)

É importante citar que a medição da memória RAM considerou a memória inicial e total do sistema para estabelecer qual foi o consumo relativo de RAM. Sendo assim, a quantificação de consumo de memória se deu ao nível do sistema operacional.

A avaliação realizada demonstrou que as duas ferramentas analisadas cumprem com seus objetivos, entregando resultados bastante satisfatórios para os processos de ETL. Contudo, a partir desta avaliação foi percebido que o Apache NiFi foi mais rápido para completar o fluxo de dados com apenas 39 minutos, em média, em razão de que essa ferramenta aplica o paralelismo interno²⁹, devido à sua arquitetura de processamento em fluxo e paralelismo interno, especialmente evidente nas operações *SplitRecord*, *QueryRecord* e *PutDatabaseRecord*. Já o Apache Airflow é totalmente sequencial, executando apenas um processo por vez. Além disso, embora seja possível habilitar o paralelismo no Apache

²⁹ Capacidade de um sistema executar múltiplas tarefas ou partes de uma mesma tarefa simultaneamente. Disponível em: <<https://bevy-cheatbook.github.io/programming/par-iter.html>>. Acesso em: 07 dez. 2025.

Airflow, essa ferramenta não consegue paralelizar a execução interna de uma única tarefa, como ocorre com o Apache NiFi.

Contudo, o Apache Airflow é mais leve, utiliza menos memória RAM e menos processamento. Já o NiFi é mais pesado e utiliza mais recursos do sistema como: RAM, CPU e mais espaço em disco.

Em se tratando da média do consumo de CPU para a execução dos fluxos ETL, o Apache Airflow se sobressai novamente, apresentando valores que variam entre 3.2 a 3.7%, contra o Apache NiFi que oscilou entre 4.0 a 5.0%, em razão da utilização dos processadores *SplitRecord*, *QueryRecord* e *PutDatabaseRecord* os quais exigem muito mais CPU devido ao uso intensivo de arquivos JSON³⁰ e *parsing* de registros.

Durante a execução dos experimentos, observou-se que o consumo de CPU permaneceu consistentemente baixo para ambas as ferramentas avaliadas, Apache NiFi e Apache Airflow. Esse comportamento pode ser explicado pelas características intrínsecas do tipo de processamento realizado, bem como pela arquitetura das ferramentas e pela natureza das operações envolvidas no *pipeline* de dados.

Nesse sentido, as médias de consumo para o Apache Airflow foram 2,29 GB para memória RAM e 3,45% para o consumo de processamento. Já para o Apache NiFi, o consumo de RAM foi de 2,36 GB e de CPU foi, 4,5%. Há picos de consumo de CPU do Apache NiFi em razão da taxa elevada no fluxo do que o Apache Airflow, que se manteve constante e com baixo consumo de CPU (em relação ao NiFi). Isso demonstra que o Airflow foi mais leve e estável na execução dos processos de ETL, do que o Apache NiFi.

O Apache NiFi tem operação mais onerosa em termos de recursos computacionais. Esse comportamento decorre, principalmente, do custo elevado das operações baseadas em JSON, da criação de estruturas temporárias em memória que servem para armazenar outros processadores, como, por exemplo, o *SplitRecord* e a JVM³¹, demandando recursos volumosos durante a execução.

Além disso, mesmo quando configurado com apenas uma *thread*³², o NiFi emprega paralelismo interno, no qual múltiplas etapas do fluxo são executadas simultaneamente. Assim, enquanto um processador como o *QueryRecord*, a consulta, transforma o primeiro fragmento de dados, o *SplitRecord* já produz o próximo, e, paralelamente, o *PutDatabaseRecord* realiza a inserção do fragmento anterior no banco de dados. Em contraste, o

³⁰ JSON (JavaScript Object Notation), sendo este, um formato leve e legível por humanos para troca de dados estruturados, baseado na sintaxe de objetos JavaScript, usado para transferir informações entre sistemas (servidor/aplicativo web) de forma eficiente. Disponível em: <https://www.alura.com.br/artigos/o-que-e-json?srsId=AfmBOoq9Hb6JShZqR6Z_aqk614uXRw88_qXmyy8QtDZa8kFJttG0YQa>. Acesso em: 07 dez. 2025.

³¹ A JVM (Java Virtual Machine) é uma máquina virtual que atua como uma camada intermediária entre o código Java e o hardware/sistema operacional, permitindo que programas escritos em Java rodem em diferentes plataformas. Disponível em: <<https://coodesh.com/blog/dicionario/o-que-e-jvm/>>. Acesso em: 07 dez. 2025.

³² thread é uma "linha de execução" dentro de um processo, permitindo que um programa faça várias coisas ao mesmo tempo. Disponível em: <<https://www.ibm.com/docs/pt-br/aix/7.3.0?topic=programming-understanding-threads-processes>>. Acesso em: 07 dez. 2025.

Airflow, executa o processamento de forma estritamente sequencial: o sistema lê um bloco de dados, processa-o, insere-o no PostgreSQL, aguarda o *commit* e somente então inicia o próximo bloco.

Torna-se evidente que, embora o Airflow seja mais leve, previsível e fácil de utilizar, o NiFi apresentou menor tempo de execução em razão de sua arquitetura de paralelismo interno. Porém, em se tratando do consumo mais baixo de recursos do sistema como, CPU e RAM, por exemplo, o Airflow se mostrou mais eficiente.

Considerando que o Apache Airflow apresentou um tempo total de execução significativamente mais elevado nos experimentos realizados, os resultados indicam que o Apache NiFi se mostrou uma alternativa mais eficiente para o cenário específico avaliado neste estudo, especialmente no que se refere à ingestão massiva de dados estruturados em um banco de dados relacional.

Em relação ao funcionamento do NiFi, o presente trabalho comprovou que ele consome mais recursos do sistema devido ao paralelismo interno e *pipelining* entre os estágios dos processadores *SplitRecord*, *QueryRecord* e *PutDatabaseRecord*. Mesmo com apenas uma tarefa por processador, diversas tarefas internas do próprio motor de funcionamento de fluxo permitem que diferentes etapas do *pipeline* sejam executadas simultaneamente. Sendo assim, o ponto mais crítico do NiFi é o alto consumo de memória RAM, comparado ao Airflow, embora em nenhuma das ferramentas ocorreu situação de estouro de memória.

No Airflow, a linguagem de programação Python oferece maior vantagem à ferramenta em razão da grande quantidade de bibliotecas de importação, documentação e fóruns que auxiliam o profissional de TI³³ na sua configuração e customização, conforme o seu objetivo. Já o NiFi apresenta pontos negativos quanto a isso, pois, comparado ao Airflow, a quantidade de exemplos práticos, tutoriais e fóruns para auxílio aos profissionais da área é bem menor.

Dessa forma, o NiFi é uma ferramenta adequada para processos de ETL com ingestão volumosa de dados diretamente em bancos de dados relacionais com suporte nativo a paralelismo interno *pipelining* e controle de vazão³⁴, permitindo alta taxa de transferência. Já o Airflow, não possui mecanismos internos de paralelização fina, o que o torna menos eficiente para ingestões de grande volume e mais adequado a processos de automação e coordenação de ETLs periódicas.

³³ Tecnologia da Informação. Disponível em: <<https://online.pucrs.br/blog/tecnologia-da-informacao>>. Acesso em: 07 dez. 2025.

³⁴ Controle de vazão (*flow control*) é o mecanismo que regula a velocidade com que os dados passam pelo fluxo, evitando sobrecarga e garantindo que cada etapa processe somente o volume que consegue suportar.

6 Conclusão

Este trabalho apresentou um comparativo de duas ferramentas gratuitas de Big Data: Apache NiFi e Apache Airflow, realizando uma comparação de eficiência em relação ao consumo de memória RAM, frequência do processador e tempo gasto para a realização do processo completo e eficiente. O contexto de Big Data apresenta muitos desafios para os processos ETL, como testes diversos e grandes volumes de dados. Assim, a principal motivação para a realização deste trabalho foi demonstrar que, através de ferramentas ETL gratuitas e a utilização de dados públicos do SUS, podem constituir fluxos ETL em DW que serão processados pelas duas ferramentas.

Os resultados obtidos são de que o Apache Airflow é uma ferramenta muito mais fácil de configurar, utilizar e possui uma quantidade grande de tutoriais e fóruns que auxiliam os profissionais da área na sua utilização.

Do ponto de vista conceitual e arquitetural, o Apache Airflow não é inerentemente sequencial, uma vez que oferece suporte à execução paralela de tarefas e DAGs independentes por meio de seus mecanismos de orquestração. Assim, em ambientes produtivos, é comum que DAGs sejam executados de forma concorrente, desde que não existam dependências explícitas entre eles e que a infraestrutura disponível suporte tal concorrência.

Entretanto, no contexto do presente estudo, a adoção de uma execução estritamente sequencial dos DAGs mostrou-se metodologicamente mais adequada, considerando os objetivos de avaliação comparativa de desempenho entre Apache Airflow e Apache NiFi. A execução sequencial permitiu estabelecer um ambiente controlado, no qual cada pipeline fosse executado de forma isolada, reduzindo interferências causadas por competição simultânea por recursos computacionais, como CPU, memória, disco e conexões com o banco de dados.

Em se tratando dos testes realizados com as duas ferramentas analisadas neste trabalho, embora a execução estritamente sequencial no Apache NiFi pudesse, à primeira vista, parecer desejável para estabelecer um ponto de comparação mais direto com o Apache Airflow, tal abordagem não é metodologicamente adequada, pois desconsidera princípios fundamentais da arquitetura e do modelo de execução do NiFi.

Forçar o NiFi a operar de maneira estritamente sequencial implicaria restringir artificialmente seu modelo de execução, eliminando mecanismos essenciais como paralelismo interno, *pipeline parallelism* e controle de vazão. Tal restrição comprometeria a validade externa do experimento, pois os resultados obtidos deixariam de representar o comportamento real da ferramenta em cenários práticos de uso. Em estudos comparativos, é metodologicamente mais apropriado avaliar cada ferramenta respeitando seu modelo operacional nativo, em vez de tentar igualá-las por meio de configurações que distorcem seu funcionamento.

Contudo, é tecnicamente possível implementar paralelismo interno dentro de uma tarefa por meio de bibliotecas Python, como *multiprocessing* ou *concurrent.futures*, essa prática não é, em geral, recomendada no contexto do Airflow, pois, o paralelismo interno

tende a reduzir a observabilidade do *workflow*, dificultar o controle de falhas e compromete mecanismos nativos da ferramenta, como reexecução seletiva (*retry*) e isolamento de tarefas. Por esse motivo, a literatura e a documentação oficial recomendam que o paralelismo seja explicitamente modelado no nível de DAGs e tarefas, mantendo o Airflow como camada de coordenação.

Além disso, na execução do presente trabalho, o Apache Airflow, embora seja uma ferramenta capaz de realizar o gerenciamento de dependências cruzadas entre DAGs diferentes, optou-se pelo isolamento para não enviesar a medição de recursos de hardware.

Nesse sentido, a realização deste trabalho contribuirá para que os profissionais da área de Tecnologia da Informação (TI), escolham qual delas é a melhor com fluxos volumosos de dados. Como trabalhos futuros, recomenda-se a ampliação da investigação para ambientes distribuídos e de maior escala, avaliando o desempenho das ferramentas em *clusters* ou contêineres orquestrados. Sugere-se, ainda, a inclusão de outras ferramentas de código aberto, testes com diferentes modelos de dados, bem como a análise do impacto de otimizações específicas (como paralelização customizada, particionamento e uso de formatos colunares) na eficiência dos processos de ETL.

7 Agradecimentos

Agradeço, antes de tudo, a Deus, por sustentar cada passo desta caminhada e renovar minhas forças nos momentos em que mais precisei.

Sou imensamente grato ao meu pai, à minha mãe e ao meu tio, que se dedicaram incansavelmente para que eu pudesse chegar até aqui.

Ao meu professor, Evandrino Gomes Barros, agradeço pelos anos de convivência, pelas experiências compartilhadas, pelos aprendizados, e por todas as lembranças que guardarei.

Por fim, deixo minha profunda gratidão a todos que acreditaram em mim e contribuíram, de alguma forma, para que minha graduação se tornasse realidade.

Referências

AGUIAR C. G. B.; FARIA JUNIOR, C. S. P. J. D. A. S. D. J. G. M. S. Avaliação do desempenho do microai atom em ambientes de recursos limitados durante ataques dos. <Disponível em: <https://congressos.ifsp.edu.br/conict/article/view/979/71>>, 2024. Acesso em: 24 nov. 2025.

ANTONIO, J. *Informática para concursos*. São Paulo: Elsevier, 2006. v. 3.ed.

BAZZI, C. L. *Introdução a banco de dados*. Curitiba: Ed. UTFPR, 2013.

CARVALHO, V. *PostgreSQL: banco de dados para aplicações web modernas*. São Paulo: Alura, 2017.

CRICKARD, P. *Engenharia de Dados com Python*. Porto Alegre: Um Livro, 2020.

ERICKSON S.; ROTHBERG, H. Big data and knowledge management: Establishing a conceptual foundation. *The Eletronic Journal of Knowledge Management*, v. 12, n. 2, p. 108–116, 2014. Acesso em 10 mai. 2025.

FERREIRA J.; MIRANDA, M. A. A. M. J. *O Processo ETL em Sistemas Data Warehouse*. São Paulo., 2010. 9f. Dissertação (Simpósio de Informática). p.

FRANZONI, D. Migração para nuvem de ferramenta de auxílio à tomada de decisão em operações de compra e venda de ativos utilizando fast api e airflow. *Monografia (Bacharelado em Engenharia de Controle e Automação)*, Florianópolis – SC, 2020.

GALDINO, N. *Big Data: Ferramentas e Aplicabilidade*. In: *SIMPÓSIO DE EXCELÊNCIA E GESTÃO EM TECNOLOGIA, 2016, on-line. Anais eletrônicos [...]*. Rio de Janeiro, 2016. Acesso em 20. Fev. 2025.

GOCHI, F. *Apache NiFi e AWS Cognito no contexto de uma cultura orientada a dados*. 2023. <<https://elogroup.com/insights/apache-nifi/>>. Acesso em: 22 jun. 2025.

GUARDIANI, J. et al. *Sistema orientado a dados para monitoração de perdas em processos industriais*. 2022. <https://www.sba.org.br/cba2022/wp-content/uploads/artigos_cba2022/paper_525.pdf>. Acesso em: 15 mai. 2025.

JESUS, F. C.; CAMARGOS, M. T. d.; SOUZA, R. C. d. Estudo comparativo de ferramentas livres de engenharia de dados: Airflow vs. nifi. *Centro Universitário Instituto de Educação Superior de Brasília*, Brasília, DF, 2023. Artigo acadêmico do curso de Engenharia/Ciência da Computação.

KIMBALL, R. *Data Warehouse Toolkit*. São Paulo: Makron Books, 1998. Tradução: Mônica Rosemberg; Revisão técnica: Ronal Stevis Cassiolato.

NETO, A. P. *PostgreSQL: Técnicas Avançadas: soluções para desenvolvedores e administradores de banco de dados*. São Paulo: Érica, 2007.

OLIVEIRA, E. L. et al. A influência da adoção do big data na análise do conhecimento da tecnologia e da propensão ao uso por gestores. *Revista Administração em Diálogo*, São Paulo, v. 24, n. 1, p. 133–151, 2022.

PAGOTTO D. P.; MARQUES, W. S. O. D. S. F. V. R. S. A. V. N. B. J. C. V. Inovação em saúde: a implementação de um data lake para armazenamento, sistematização e disponibilização de dados em saúde no brasil. <*Disponível em: https://revistas.usp.br/incid/article/view/213345*>, 2024. Acesso em: 22 nov. 2025.

SILVA, D. G. *Criação de um data warehouse para dados públicos de atendimentos ambulatoriais do SUS*. 2024. Monografia (MBA em Inteligência Artificial e Big Data) – Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2024.

SILVA, M.; SARTORI, M. Data warehouse: a vantagem da modelagem dimensional de dados. In: *Encontro Científico Cultural Interinstitucional*. Cascavel: [s.n.], 2015. p. 1–10.

SOUZA, A. C.; AMARAL, H. R.; LIZARDO, L. E. O. *PostgreSQL: uma alternativa para sistemas gerenciadores de banco de dados de código aberto*. 2011. <<http://www.periodicos.letras.ufmg.br/index.php/ueadsl/article/viewFile/2896/2855>>. Acesso em: 18 nov. 2025.

SOUZA, M. V. C. *Data warehouse e ETL para dados da saúde pública: uma aplicação prática*. 2020. Monografia (Bacharelado em Ciências da Computação) – Instituto de Matemática e Estatística, Universidade de São Paulo, São Paulo, 2020, 70p.

SOUZA, V. G. *Estudo de performance do Apache Hive em sistemas de data warehouse em nuvem*. 2021. <https://www.cin.ufpe.br/~tg/2021-1/tg_EC/TG_vgs2.pdf>. Acesso em: 07 abr. 2025.

TOLEDO D. F.; CORREIA, R. C. M. S. C. T. S. Descoberta de conhecimento em base de dados: uma análise de ferramentas etl. <*Disponível em: https://ojs.revistagesec.org.br/secretariado/article/view/4427*>, 2025. Acesso em: 24 nov. 2025.

VASCONCELOS, E. M. *Demonstrando a utilização de ETL com Apache NiFi: um estudo de caso com despesas públicas federais*. Dissertação (Trabalho de Conclusão de Curso) — Instituto Federal do Espírito Santo – IFES, 2024. Acesso em: 2 dez. 2025. Disponível em: <<https://repositorio.ifes.edu.br/handle/123456789/5591>>.

YANG, Y.; VETTER, J. S. *Modeling the Linux page cache for accurate simulation of data-intensive applications*. 2021. <<https://arxiv.org/abs/2101.01335>>. Acesso em: 3 dez. 2025.